

AI Assisted Coding

Assignment 10.3

Name: P. Vineeth Kumar

Hall ticket no: 2303A51256

Batch no: 19

Task 1: Variable Naming Issues

Prompt:

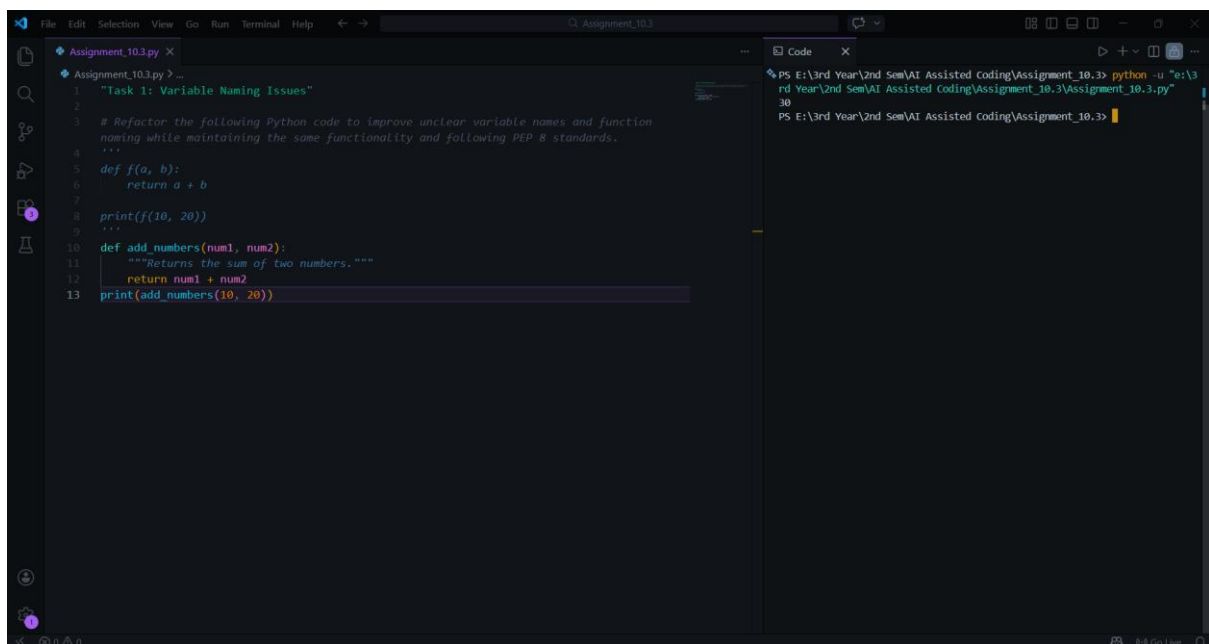
Refactor the following Python code to improve unclear variable names and function naming while maintaining the same functionality and following PEP 8 standards.

```
def f(a, b):
```

```
    return a + b
```

```
print(f(10, 20))
```

Code & Output:



The screenshot shows a code editor with two panes. The left pane displays the refactored Python code, and the right pane shows the command prompt output.

```
1 "Task 1: Variable Naming Issues"
2
3 # Refactor the following Python code to improve unclear variable names and function
4 # naming while maintaining the same functionality and following PEP 8 standards.
5 ...
6 def add_numbers(num1, num2):
7     """Returns the sum of two numbers."""
8     return num1 + num2
9 ...
10 print(add_numbers(10, 20))
11 ...
12 def first_number(num1, num2):
13     """Returns the first number of two numbers."""
14     return num1
15 ...
16 print(first_number(10, 20))
```

The right pane shows the command prompt output:

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_10.3> python -u "e:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_10.3\Assignment_10.3.py"
30
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_10.3>
```

Explanation:

The original code uses unclear names such as *f*, *a*, and *b*, which do not describe their purpose. The AI-refactored version replaces them with meaningful identifiers like *add_numbers*, *first_number*, and *second_number*. This improves readability and makes the function's purpose immediately clear. The new version also follows PEP 8 naming conventions, enhancing maintainability without changing functionality.

Task 2: Missing Error Handling

Prompt:

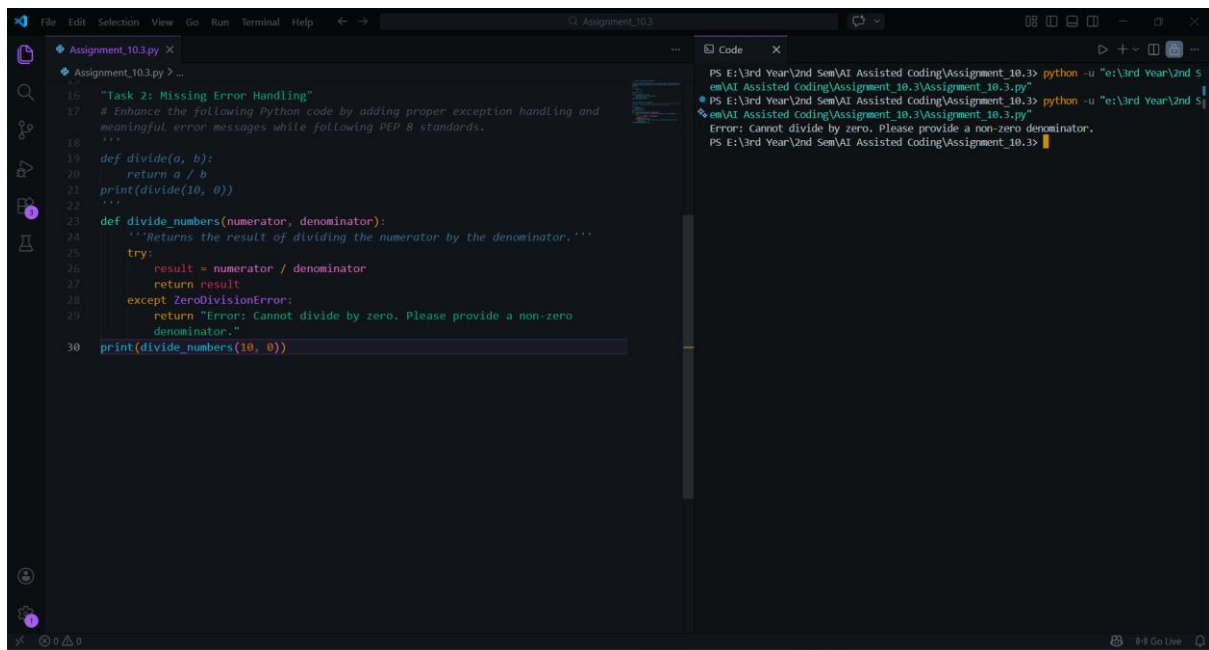
Enhance the following Python code by adding proper exception handling and meaningful error messages while following PEP 8 standards.

```
def divide(a, b):
```

```
    return a / b
```

```
print(divide(10, 0))
```

Code & Output:



The screenshot shows a code editor with a file named 'Assignment_10.3.py'. The code defines two functions: 'divide(a, b)' and 'divide_numbers(numerator, denominator)'. The 'divide' function is the original code, and 'divide_numbers' is the enhanced version with a try-except block for ZeroDivisionError. The terminal output shows the command 'python -u "e:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_10.3.py"' and the resulting error message: 'Error: Cannot divide by zero. Please provide a non-zero denominator.'

```
File Edit Selection View Go Run Terminal Help ← → Assignment_10.3
Assignment_10.3.py ×
Assignment_10.3.py > ...
16 "Task 2: Missing Error Handling"
17 # Enhance the following Python code by adding proper exception handling and
18 # meaningful error messages while following PEP 8 standards.
19 ...
20 def divide(a, b):
21     return a / b
22     print(divide(10, 0))
23 ...
24 def divide_numbers(numerator, denominator):
25     """Returns the result of dividing the numerator by the denominator."""
26     try:
27         result = numerator / denominator
28         return result
29     except ZeroDivisionError:
30         return "Error: Cannot divide by zero. Please provide a non-zero
31         denominator."
32     print(divide_numbers(10, 0))

Code ×
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_10.3> python -u "e:\3rd Year\2nd S
em\AI Assisted Coding\Assignment_10.3.py"
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_10.3> python -u "e:\3rd Year\2nd S
em\AI Assisted Coding\Assignment_10.3.py"
Error: Cannot divide by zero. Please provide a non-zero denominator.
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_10.3>
```

Explanation:

The original code does not handle division by zero, which causes a runtime error. The AI-enhanced version introduces a try-except block to handle this scenario gracefully.

Meaningful variable names improve clarity. This improves robustness, prevents crashes, and ensures better user experience. Error handling is essential for writing production-quality software.

Task 3: Student Marks Processing System

Prompt:

Refactor the given student marks program to improve readability, follow PEP 8 standards, include meaningful variable names, modular functions, documentation, and basic validation.

```
marks=[78,85,90,66,88]
t=0
for i in marks:
    t=t+i
a=t/len(marks)
if a>=90:
    print("A")
elif a>=75:
    print("B")
elif a>=60:
    print("C")
else:
    print("F")
```

Code & Output:

```

12  """Task 3: Student Marks Processing System"""
13  # Refactor the given student marks program to improve readability, follow PEP 8
14  # standards, include meaningful variable names, modular functions, documentation, and
15  # basic validation.
16  marks=[78,85,90,66,88]
17  t=0
18  for i in marks:
19      t=t+i
20  a=t/len(marks)
21  if a>=90:
22      print("A")
23  elif a>=75:
24      print("B")
25  elif a>=60:
26      print("C")
27  else:
28      print("F")
29
30  def calculate_average(marks):
31      """Calculates the average of a list of marks."""
32      total_marks = sum(marks)
33      average_mark = total_marks / len(marks)
34      return average_mark
35
36  def determine_grade(average_mark):
37      """Determines the grade based on the average mark."""
38      if average_mark >= 90:
39          return "A"
40      elif average_mark >= 75:
41          return "B"
42      elif average_mark >= 60:
43          return "C"
44      else:
45          return "F"
46
47  def main():
48      """Main function to process student marks and determine the grade."""
49      marks = [78, 85, 90, 66, 88]
50      average_mark = calculate_average(marks)
51      grade = determine_grade(average_mark)
52      print(f"The average mark is: {average_mark:.2f}")
53      print(f"The grade is: {grade}")
54
55  if __name__ == "__main__":
56      main()

```

Output:

```

PS E:\3rd Year\2nd Sem\AI Assisted coding\Assignment_10.3> python -u "E:\3rd Year\2nd Sem\AI Assisted coding\Assignment_10.3\Assignment_10.3.py"
The average mark is: 81.40
The grade is: B
PS E:\3rd Year\2nd Sem\AI Assisted coding\Assignment_10.3>

```

Explanation:

The original code lacks structure, meaningful variable names, and documentation. The AI-refactored version modularizes the logic into a function, improves variable naming, and follows PEP 8 formatting. The use of built-in `sum()` improves efficiency and readability. This version is more maintainable and reusable.

Task 4: Adding Docstrings and Inline Comments

Prompt:

Enhance the factorial function by adding a proper docstring and meaningful inline comments.

```
def factorial(n):
    result = 1
    for i in range(1,n+1):
        result *= i
    return result
```

Code & Output:

The screenshot shows a code editor with a file named 'Assignment_10.3.py'. The code defines a factorial function with a docstring and inline comments. The terminal window shows the command 'python -u "e:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_10.3.py"' and the output '120'.

```
72
73 "Task 4: Adding Docstrings and Inline Comments"
74 # Enhance the factorial function by adding a proper
  docstring and meaningful inline comments.
75 '''def factorial(n):
76     result = 1
77     for i in range(1,n+1):
78         result *= i
79     return result'''
80 def factorial(n):
81     '''Returns the factorial of a given number n.'''
82     result = 1 # Initialize result to 1, as factorial of
      0 is 1
83     for i in range(1, n + 1): # Loop from 1 to n
      (inclusive)
84         result *= i # Multiply result by the current
      number i
85     return result # Return the final factorial value
86 # Example usage
87 print(factorial(5)) # Output: 120
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_10.3> python -u "e:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_10.3.py"
120
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_10.3>
```

Explanation:

The AI-enhanced function adds a docstring describing the function's purpose and parameter. Inline comments explain each logical step. Variable naming is improved for clarity. This enhances readability and documentation quality.

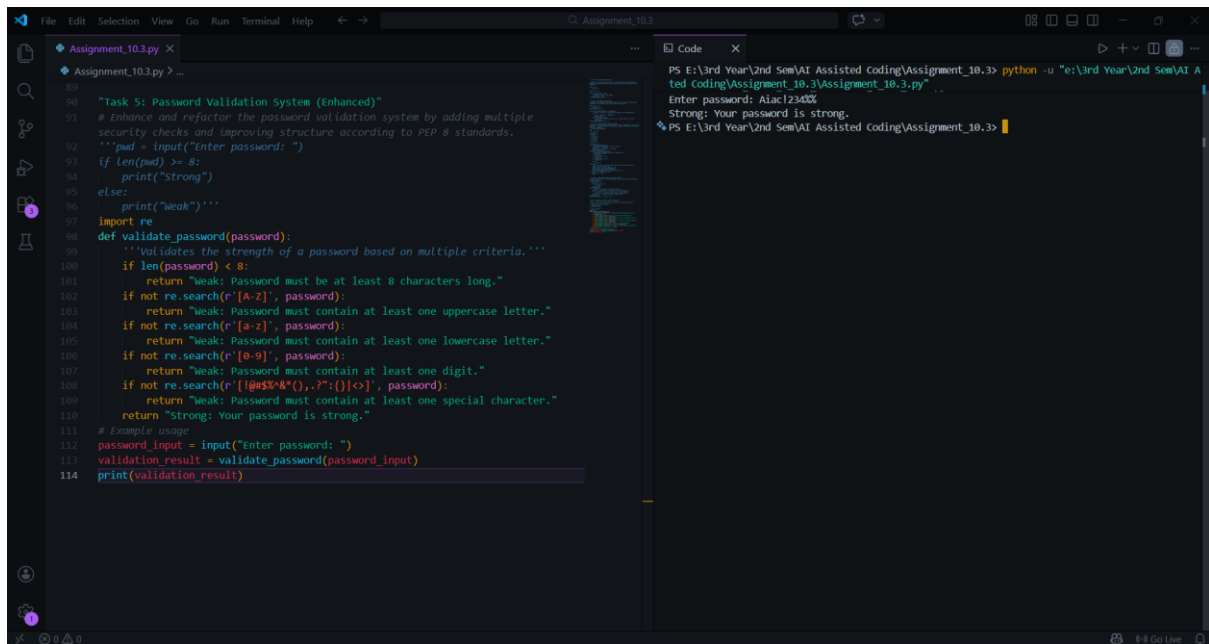
Task 5: Password Validation System (Enhanced)

Prompt:

Enhance and refactor the password validation system by adding multiple security checks and improving structure according to PEP 8 standards.

```
pwd = input("Enter password: ")
if len(pwd) >= 8:
    print("Strong")
else:
    print("Weak")
```

Code & Output:



```
File Edit Selection View Go Run Terminal Help Assignment_10.3
Assignment_10.3.py x
Assignment_10.3.py > ...
90 """Task 5: Password Validation System (Enhanced)"""
91 # Enhance and refactor the password validation system by adding multiple
92 # security checks and improving structure according to PEP 8 standards.
93 """pwd = input("Enter password: ")
94 if len(pwd) >= 8:
95     print("Strong")
96 else:
97     print("Weak")"""
98 import re
99 def validate_password(password):
100     """Validates the strength of a password based on multiple criteria."""
101     if len(password) < 8:
102         return "Weak: Password must be at least 8 characters long."
103     if not re.search(r'[A-Z]', password):
104         return "Weak: Password must contain at least one uppercase letter."
105     if not re.search(r'[a-z]', password):
106         return "Weak: Password must contain at least one lowercase letter."
107     if not re.search(r'[0-9]', password):
108         return "Weak: Password must contain at least one digit."
109     if not re.search(r'[!@#$%^&*()_~?":{}|<>]', password):
110         return "Weak: Password must contain at least one special character."
111     return "Strong: Your password is strong."
112 # Example usage
113 password_input = input("Enter password: ")
114 validation_result = validate_password(password_input)
115 print(validation_result)
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_10.3> python -u "e:\3rd Year\2nd Sem\AI A
ted Coding\Assignment_10.3\Assignment_10.3.py" ..
Enter password: Alac1234QX
Strong: Your password is strong.
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_10.3>
```

Explanation:

The enhanced program introduces multiple password security rules including uppercase, lowercase, digits, and special characters. The logic is modularized into a function with documentation. Compared to the original version, the new program is more secure, readable, and maintainable.

Final Conclusion:

This lab demonstrated how AI-assisted coding tools can be effectively used for automated code review and quality enhancement. AI suggestions helped improve variable naming, added proper error handling, enhanced documentation, strengthened password security, and ensured compliance with PEP 8 coding standards. The refactored programs became more readable, maintainable, and robust compared to their original versions. However, human evaluation remains essential to validate AI-generated improvements and ensure they align with software engineering best practices.